
Stage Five: Requirements Revision Log

Author: Madhav Mehta **PRD v1.0 → v2.0**

Section one: revision summary

Revision	What changed	What triggered it	Severity
R-01	The routing signal: from intent-classifier confidence to a purpose-built calibrated automation-readiness score	Calibration measurement on day four of week two	Substantive — it changed a component
R-02	The threshold: from 0.80 chosen by convention to 0.45 chosen by cost minimisation, with published sensitivity	Follows from R-01; the old number was on a different scale and meant nothing on the new one	Substantive
R-03	Private data: from redact-and-send to block-and-escalate	Reading the Governance Framework against the implementation	Substantive
R-04	New NFR-06: cross-group variation under 5 points, measured on every run	The fairness audit found a 20-point gap that v1 had no requirement to detect	Substantive
R-05	New FR-09: escalations must carry a handover brief	Daniel's interview, under-served by v1	Moderate
R-06	New FR-10: kill switch	Governance Framework §5; v1 had none	Moderate
R-07	Router rule added: a never-automate intent scoring close to the top prediction vetoes automation	A test failure	Moderate

Section two: the changes

R-01 — The routing signal

v1.0 said: route on the intent classifier's probability; auto-respond at or above 0.80.

What happened. The day-four unattended run worked. The numbers looked respectable. But the Evaluation Framework asks for a calibration table, and producing one showed something I had not expected.

Out-of-fold intent accuracy was **99.8%** — the classifier was almost never wrong. Yet observed accuracy sat at or near 100% in every confidence band from 0.1 upward, while the confidence it stated ranged from 0.14 to 0.82. Expected calibration error: **39.79 points**, against a requirement of five.

With 22 intents and near-uniform competition, logistic regression spreads probability thinly. It would report 0.08 for a prediction that was correct. A threshold of 0.80 on that distribution escalated almost everything; a lower one let almost everything through. Neither was expressing a judgement about the ticket — the number simply did not carry the meaning the requirement assumed.

Worse, sweeping it from 0.05 to 0.80 moved automation precision by **under two points**. The signal was not merely miscalibrated, it was close to irrelevant to the decision it was gating. That makes sense in hindsight: the discovery analysis shows `answerable_from_docs` ranging from 0% to 96% *across* intents, so "can this be answered automatically" is a property the intent confidence does not capture.

v2.0 says: the threshold applies to a calibrated automation-readiness score — a separate binary model trained on `expected_route` and wrapped in isotonic calibration, answering directly the question the router needs answered. Its expected calibration error is **4.24 points**, inside the requirement. Intent still drives the never-automate policy rules; it no longer drives the threshold.

What it cost: roughly seven hours, most of week two's contingency.

R-02 — The threshold

0.80 was inherited from the illustrative figure in the Project Brief. It was never derived. On the new score it would also have been meaningless, since the two numbers are on different scales.

v2.0 selects $T = 0.45$ by minimising modelled cost using figures taken from the discovery interviews — Marcus's "an escalation costs us roughly four times what a resolved one costs" — subject to never automating a ticket flagged `must_not_auto_respond`.

The cost the client did not quantify is the cost of a wrong automated answer. Rather than pick one quietly, it is a parameter with a published sensitivity analysis, and the analysis carries an uncomfortable result: the choice is stable between assumed costs of 8 and 12, but at 20 the optimum jumps to 0.90 and automation collapses to 3.6%. **The threshold is not robust to an assumption CloudServe has never been asked to make.** That is in the report as a question for the client rather than buried.

R-03 — Private data

v1 redacted API keys and credit card numbers and sent the response. The Governance Framework says block and escalate, never redact and send, and it is right: redaction hides the fault while leaving it in place. A response containing another customer's identifier means something upstream is wrong, and masking the identifier removes the evidence while preserving the defect.

R-04 — Cross-group variation

v1 had no fairness requirement. The Governance Framework asks for under five points of variation between customer groups, and measuring it found **20 points** by customer tier — standard customers resolved at 66.67% against 86.67% for business.

Adding the requirement does not fix the gap. It means the gap is measured on every run and cannot be shipped unnoticed, which is the necessary first step.

R-07 — The runner-up veto

Found by a test, not by inspection. "Could you add the ability to schedule recurring bulk exports?" classified as `billing_query` at 0.080 with `feature_request` second at 0.067. On the top prediction alone it would have been auto-answered — and a feature request has no documented answer to give; the corpus marks every one `answerable_from_docs: false`.

Keying a safety rule to the single highest-scoring class assumes a separation the classifier does not provide. Now, when a never-automate intent scores within 20% of the top prediction, the ticket escalates and says why.

Section three: assumptions that turned out to be wrong

v1 assumption	What actually happened	Consequence
A confidence score from a classifier is calibrated enough to threshold on	Off by 39.79 points	The entire routing signal was replaced
High classification accuracy implies good routing	99.8% accurate and still nearly useless for routing	Accuracy and usefulness are different properties; I had been treating them as one
Non-fluent customers get worse outcomes, per Sofia	They do slightly <i>better</i> on the historic record	A special path was not built. The real gap was by tier, which nobody mentioned
A guardrail that redacts is a guardrail	It is a way of hiding a fault	Changed to block and escalate
Zero guardrail activations in a run is a good result	It means the corpus has no attacks in it, and proves nothing	Added ten probes through the same pipeline after every run
A semantic index will beat keyword search, per Ines	Measured identical: 89.08% / 85.15% / MRR 0.8711 on all tiers	The claim is reported as unverified rather than asserted

Section four: what I decided not to change

Considered	Decided	Why
Recalibrating the intent classifier instead of replacing the signal	Not done	It would fix calibration but not relevance. A well-calibrated estimate of the wrong quantity is still the wrong quantity
Tier-aware thresholds to close the fairness gap	Not done	It would reduce the measured variation by treating customers differently on the basis of what they pay, which is the thing Ravi objects to. Reporting the gap honestly is better than engineering the metric. Recorded as the first piece of future work
Dropping the local generation path once a provider was available	Not done	It is what makes A11 real rather than aspirational, and the entire recorded evaluation ran through it
Making Chroma a hard dependency	Not done	A1 is a gate. A clean checkout must not fail on an optional 2 GB download
Automating billing queries, which Daniel distrusts	Automated	The corpus flags 0 of 24 billing tickets as must-not-automate, and 87.5% are documented. Daniel's concern is about commitments, which the guardrail blocks directly. Documented as a deliberate disagreement with a stakeholder

Section five: reflection

The most useful thing I did was run the full set unattended on day four of week two rather than day five. Everything of substance in this log came out of that one afternoon, and a day's margin was the difference between finding the calibration problem and shipping it.

The mistake underneath the biggest revision was treating a number as meaningful because it was in the range 0 to 1 and had "confidence" in its name. I had built a threshold on it, written a requirement around it, and justified a figure of 0.80 — none of which involved checking whether it measured anything. The Evaluation Framework asks for a calibration table almost in passing, as one requirement among many, and producing it invalidated a component. I would now treat "plot the calibration curve" as something you do the day a score enters a decision path, not the week you write the evaluation.

The second thing I got wrong was subtler. Sofia's account of non-fluent customers was vivid, specific and came from the person closest to the work, and I believed it. The data contradicted it. Had I not checked, I would have built a special handling path for a problem that does not exist and missed the tier gap, which is real, twenty points wide, and which nobody in five interviews mentioned. The lesson is not that stakeholders are unreliable — it is that the things nobody says are where the unexamined problems live, because everybody sees only their own part of the operation.

What I am least comfortable with is the fairness gap. I found it, I measured it, I wrote a requirement that will catch it on every run, and I did not fix it. The honest reason is that the fix I could reach — tier-aware thresholds — would improve the number by explicitly treating customers differently according to what they pay, and I could not defend that to Ravi. The better fix is to understand *why* standard-tier tickets are harder, which needs work I did not have the time for. Leaving a known 20-point gap in a system that is otherwise ready is the part of this submission I would want to address first.